WIRESHARK NETWORK TRAFFIC ANALYSIS

Tools	Kali Linux · Nmap · Wireshark · Windows PowerShell
Lab network	Private IPv4 subnet 10.0.0.0/24
Key services	ICMP · TCP · HTTP · Windows network services
Evidence	Original screenshots captured during the practical lab

1. Project overview

This project documents a small, controlled network analysis exercise using Kali Linux and a Windows host. The goal was to observe how hosts communicate, identify listening services, capture packets, interpret TCP connection setup, and troubleshoot HTTP connectivity using Wireshark and command-line tools.

The work combines network discovery with packet-level validation. Nmap was used to enumerate selected services on a lab target; ping and Wireshark were used to examine ICMP; and a simple HTTP test service on TCP port 8000 was accessed from Windows and inspected in the packet capture.

2. Lab environment and observed addressing

Kali Linux: 10.0.0.6/24 on eth0. **Windows test host:** 10.0.0.5 (as shown in the ICMP and HTTP test evidence). **Additional scanned host:** 10.0.0.4 (the Nmap service scan target). **HTTP service:** 10.0.0.6:8000, accessed from Windows.

Important: the screenshots show more than one target address. The Nmap scan was run against 10.0.0.4, while the ICMP and HTTP tests shown later involve 10.0.0.5 and 10.0.0.6. These addresses are intentionally kept distinct in this report rather than being assumed to represent the same machine.

3. Objectives

• Discover reachable hosts and identify exposed TCP services in an authorised lab.

• Capture and interpret ICMP echo requests and replies.

• Observe the TCP three-way handshake and connection teardown.

• Inspect an HTTP GET request and server response on TCP port 8000.

• Use packet evidence to investigate a later failed TCP connectivity test.

• Produce clear findings, limitations and practical security recommendations.

4. Tools and commands used

Nmap service discovery

```
nmap -sS -sV 10.0.0.4
```

The scan output identified the host as up and reported open TCP services including 135, 139, 445, 3389, 5985 and 7070. Service/version identification is indicative and should be validated before making asset or vulnerability conclusions.

Confirm Kali interface address

```
hostname -i
ip addr show eth0
```

ICMP reachability test

```
ping -c 4 10.0.0.5
```

HTTP request from Windows PowerShell

```
Invoke-WebRequest -Uri http://10.0.0.6:8000/ -UseBasicParsing
```

TCP port connectivity check

```
Test-NetConnection 10.0.0.6 -Port 8000
```

The PowerShell syntax shown is appropriate for Windows PowerShell. The successful HTTP request and later failed TCP test were captured at different points in the exercise.

5. Wireshark analysis

The following display filters were used or are suitable for reproducing the analysis:

Display filter	Purpose
icmp	Show ICMP echo requests and replies.
tcp.port == 8000	Isolate TCP conversations involving the test HTTP port.
tcp	Show TCP traffic for handshake and teardown analysis.
http	Show decoded HTTP requests and responses when the traffic is unencrypted.
ip.addr == 10.0.0.5	Limit packets to traffic involving the Windows test host.
tcp.flags.reset == 1	Find TCP reset packets, if present in the capture.

ICMP: The capture shows echo requests from 10.0.0.6 to 10.0.0.5 and echo replies returning to 10.0.0.6. The replies indicate that the destination responded to the ping probes during the capture.

TCP: For the port 8000 connection, the client sends SYN, the server responds with SYN/ACK, and the client completes the handshake with ACK. This is the normal TCP connection establishment sequence.

HTTP: The capture shows a GET request and an HTTP 200 OK response. A separate browser request for /favicon.ico receives 404 File not found. This indicates the icon resource was absent; it does not mean the main test page failed.

Connectivity troubleshooting: A later Test-NetConnection attempt to 10.0.0.6:8000 reports a failed TCP connection. Possible causes include the test web server no longer listening, a firewall rule, a service restart, or a change in lab state. The screenshot alone does not identify which cause occurred.

6. Findings and interpretation

• The Kali interface was configured with 10.0.0.6/24.

• ICMP communication between 10.0.0.6 and 10.0.0.5 was successful during the ping test.

• The Nmap scan of 10.0.0.4 identified multiple open Windows-associated services. Open ports increase the available attack surface and should be justified and restricted.

• The HTTP test service on 10.0.0.6:8000 returned HTTP 200 OK to a Windows PowerShell request.

• Wireshark exposed the TCP handshake and HTTP exchange, allowing the application-layer event to be correlated with transport-layer packets.

• The later failed TCP test demonstrates the value of capturing and comparing network state over time.

7. Security recommendations

• Disable unnecessary services and close unused listening ports.

• Restrict SMB (TCP 445), RPC, RDP and management interfaces to trusted administrative networks.

• Use host firewalls and network segmentation to limit which systems can reach management services.

• Prefer HTTPS over HTTP for real applications because HTTP traffic is not encrypted and can expose content to packet capture.

• Keep Windows and Linux systems patched and review service configurations regularly.

• Retain packet captures only when needed, and remove credentials, personal data and unrelated traffic before publishing them.

8. Limitations

This is a small training-lab exercise, not a full vulnerability assessment or penetration test. Nmap service fingerprints may be imperfect; a port being open does not prove that a vulnerability exists. The screenshots do not establish why the later TCP test failed. No claims are made about production systems or real-world compromise.

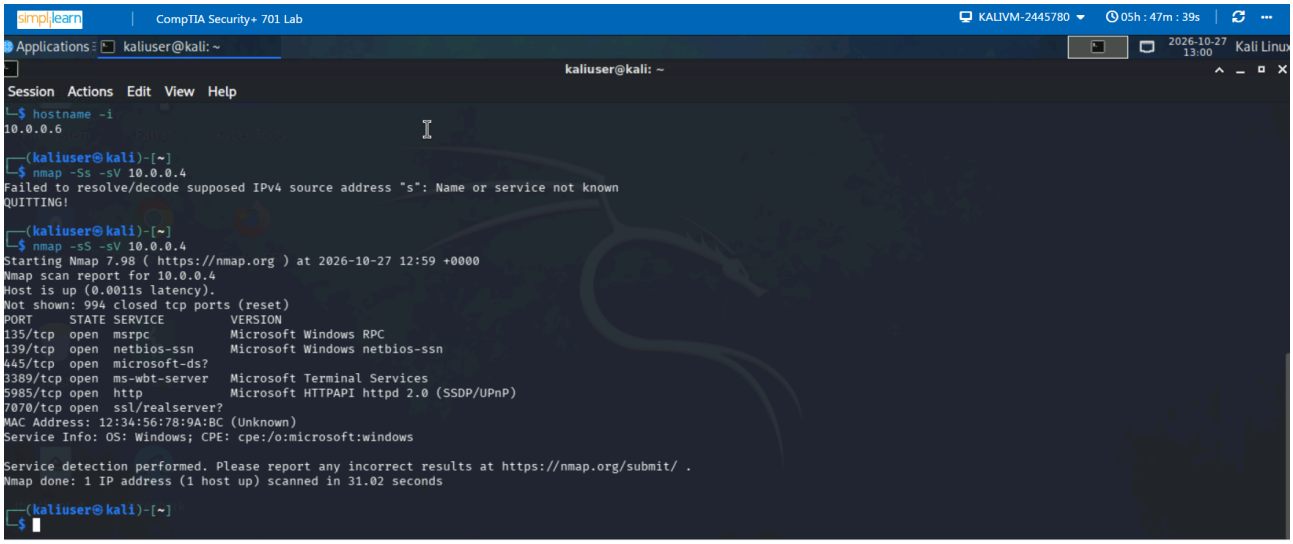
9. Conclusion

The exercise demonstrated a practical workflow for network troubleshooting: identify the host and address, test basic reachability, discover exposed services, capture traffic, filter relevant packets, and interpret the protocol exchange. It also showed how packet-level evidence can help distinguish an application response from a later connectivity failure.

Appendix A — Screenshot evidence

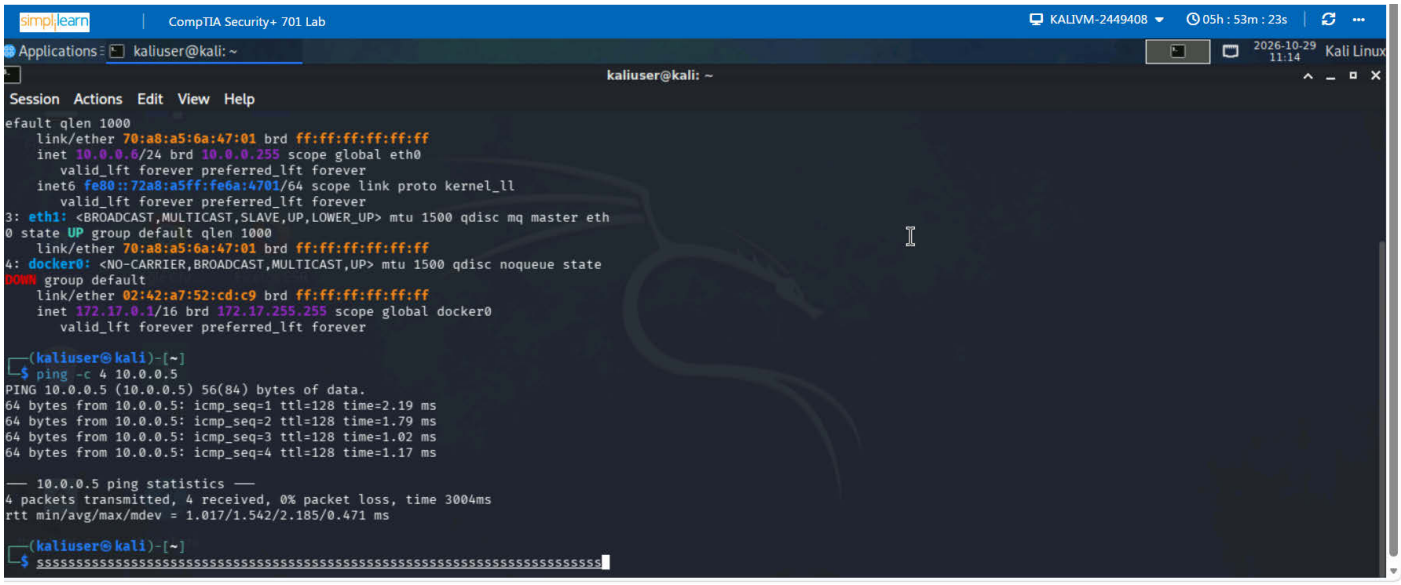
The screenshots below are included as evidence from the practical exercise.

Evidence 01: Nmap service discovery



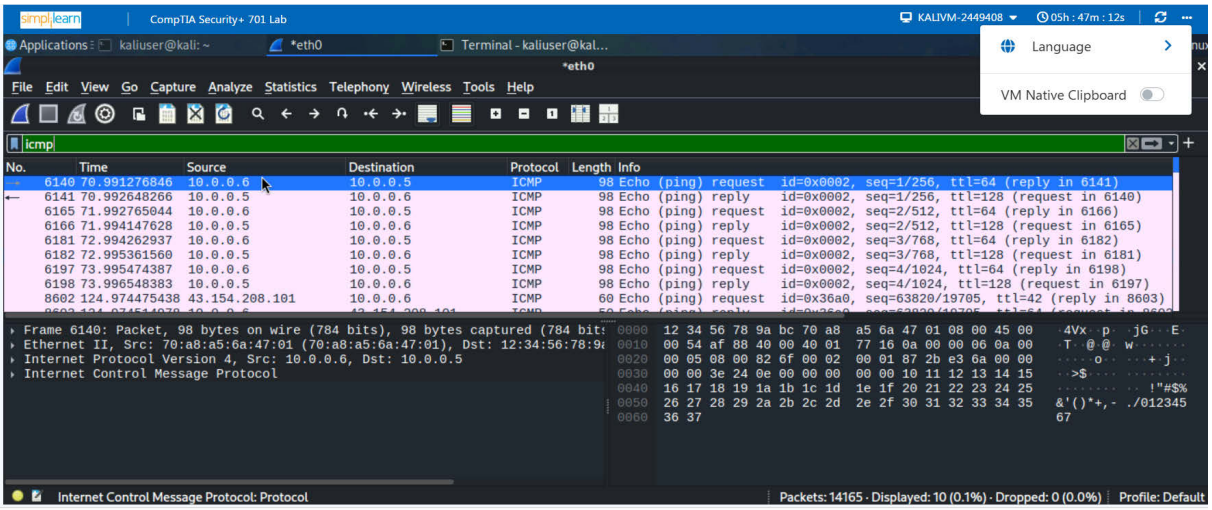
A SYN scan and service-version detection against 10.0.0.4 identified several exposed Windows-related services, including RPC (135), NetBIOS (139), SMB (445), and RDP (3389). The screenshot also shows HTTP-related services on ports 5985 and 7070. Treat these as observations from this training lab, not a recommendation to expose them.

Evidence 02: Network interface and addressing



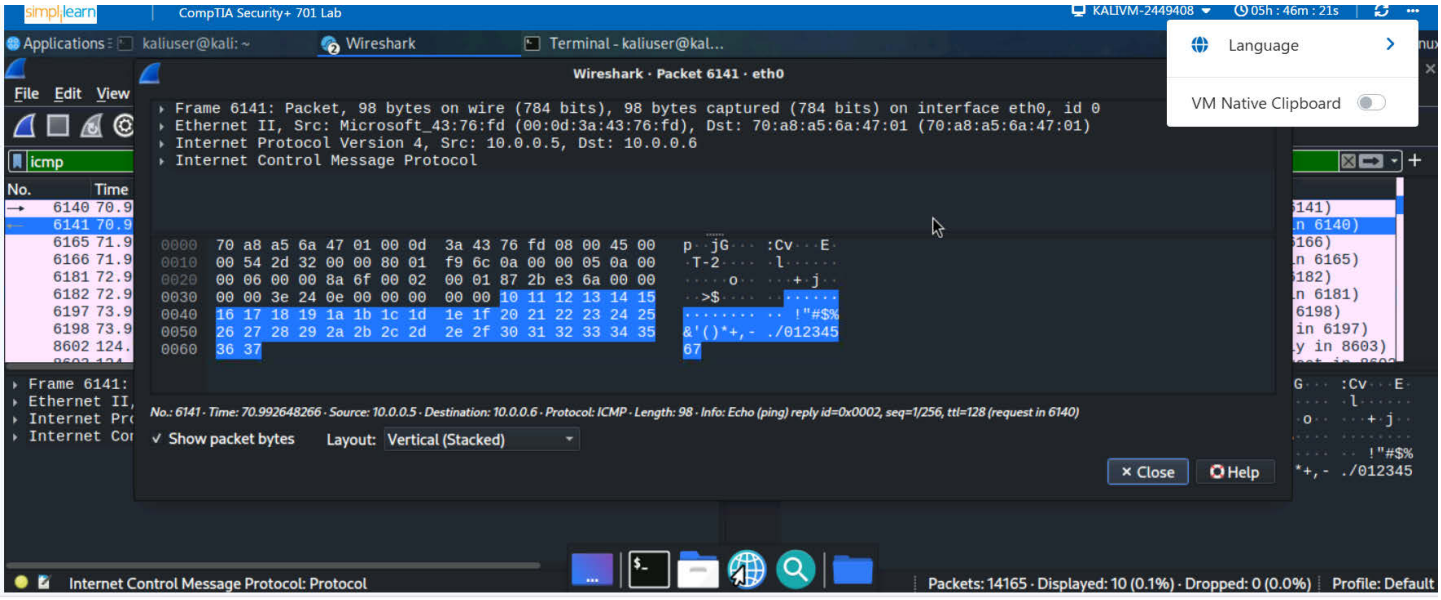
The Kali host reports 10.0.0.6/24 on eth0. This establishes the source system's lab address and subnet for the connectivity tests.

Evidence 03: ICMP echo request and reply



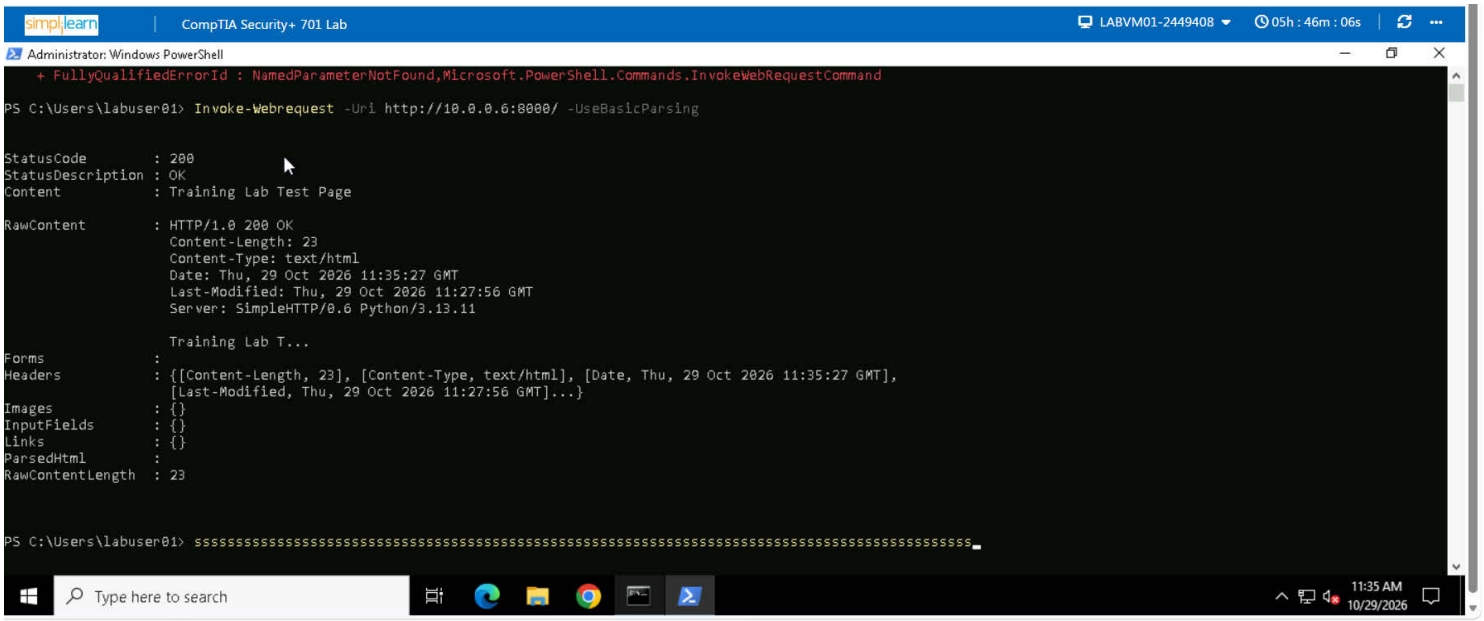
Wireshark's icmp display filter shows echo requests from 10.0.0.6 to 10.0.0.5 and corresponding replies. The exchange confirms ICMP reachability between these two lab hosts at the time of capture.

Evidence 04: ICMP packet detail and payload



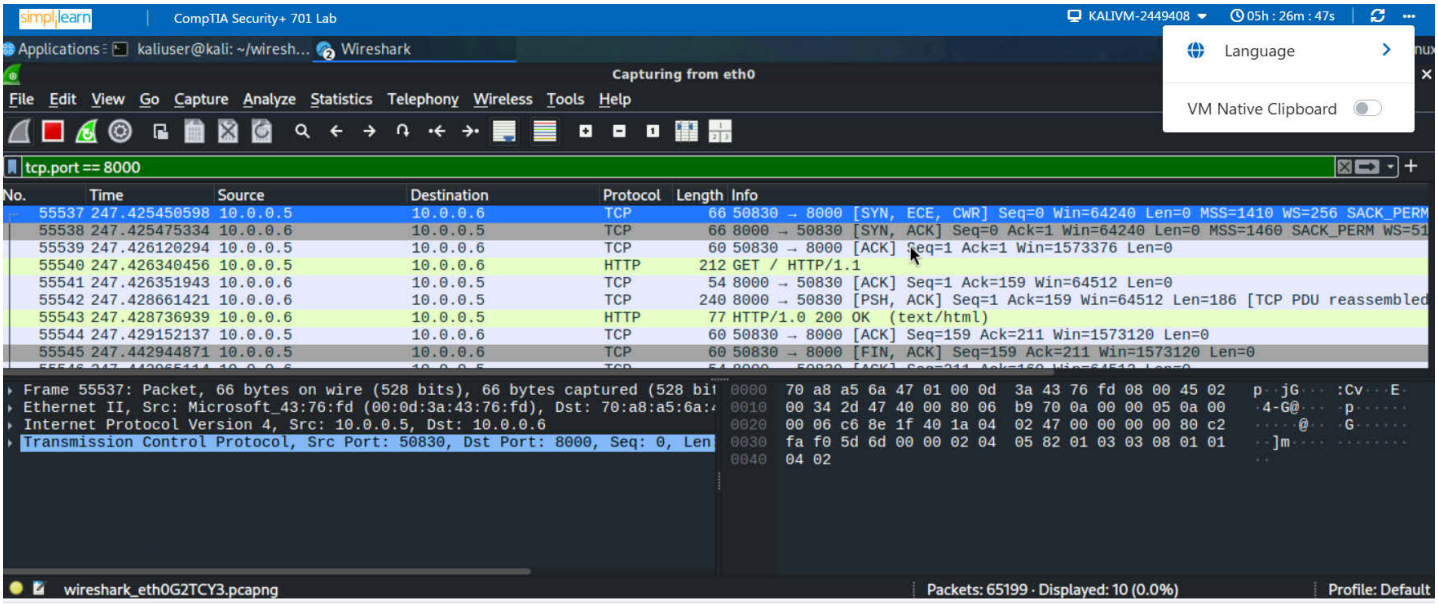
The packet detail view exposes the Ethernet, IPv4 and ICMP layers. The selected packet is an ICMP echo reply from 10.0.0.5 to 10.0.0.6; the bytes pane displays the captured packet data.

Evidence 05: HTTP response from the test web service



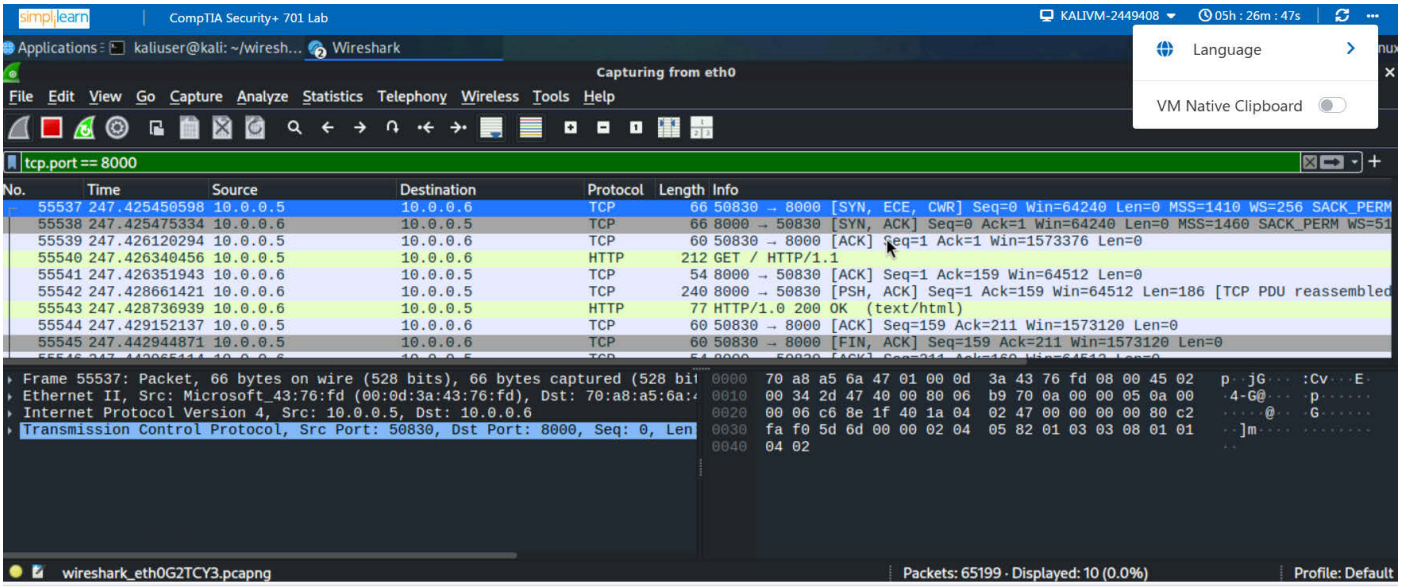
Windows PowerShell uses Invoke-WebRequest to request http://10.0.0.6:8000/. The returned status is 200 (OK), with the body 'Training Lab Test Page'. This verifies that the HTTP service responded successfully.

Evidence 06: TCP connection establishment



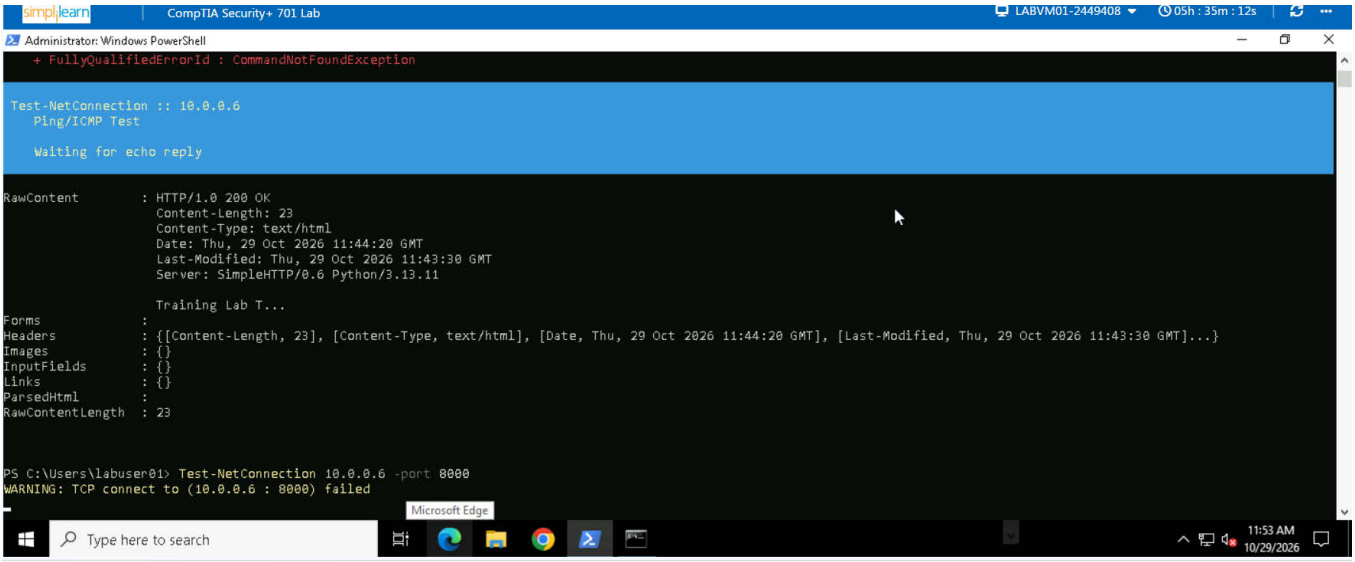
The tcp.port == 8000 filter shows the TCP three-way handshake: SYN from 10.0.0.5 to 10.0.0.6, SYN/ACK in the reverse direction, then ACK from the client. An HTTP GET follows.

Evidence 07: HTTP request and response in Wireshark



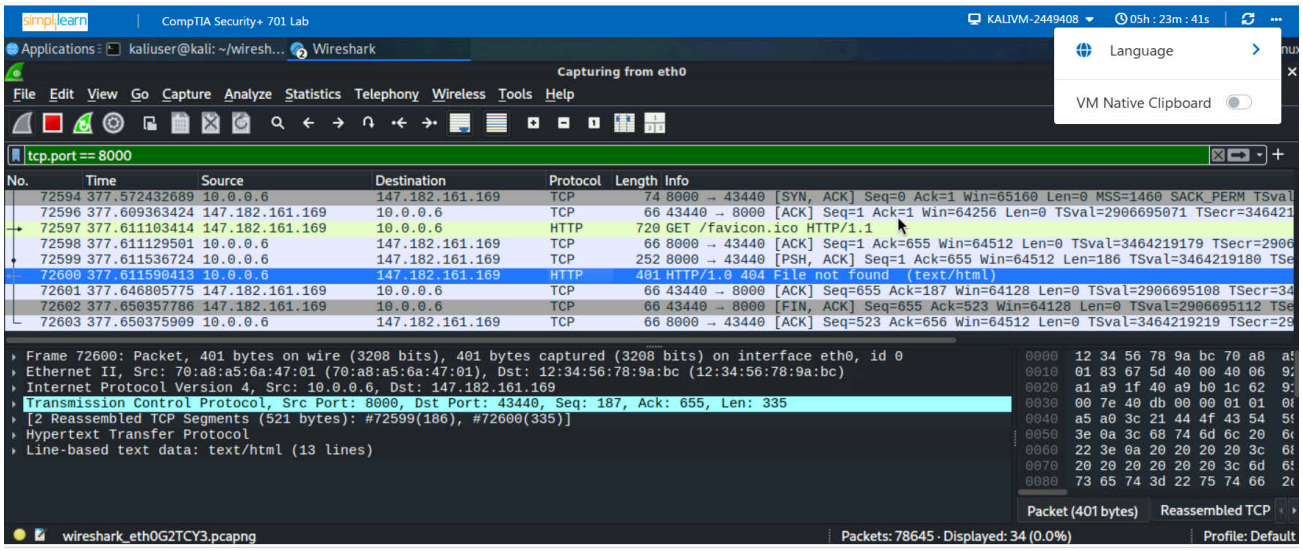
The capture shows an HTTP GET request to the service on port 8000 and an HTTP/1.0 200 OK response. Wireshark also shows TCP acknowledgements and connection teardown packets.

Evidence 08: Connectivity test to TCP port 8000



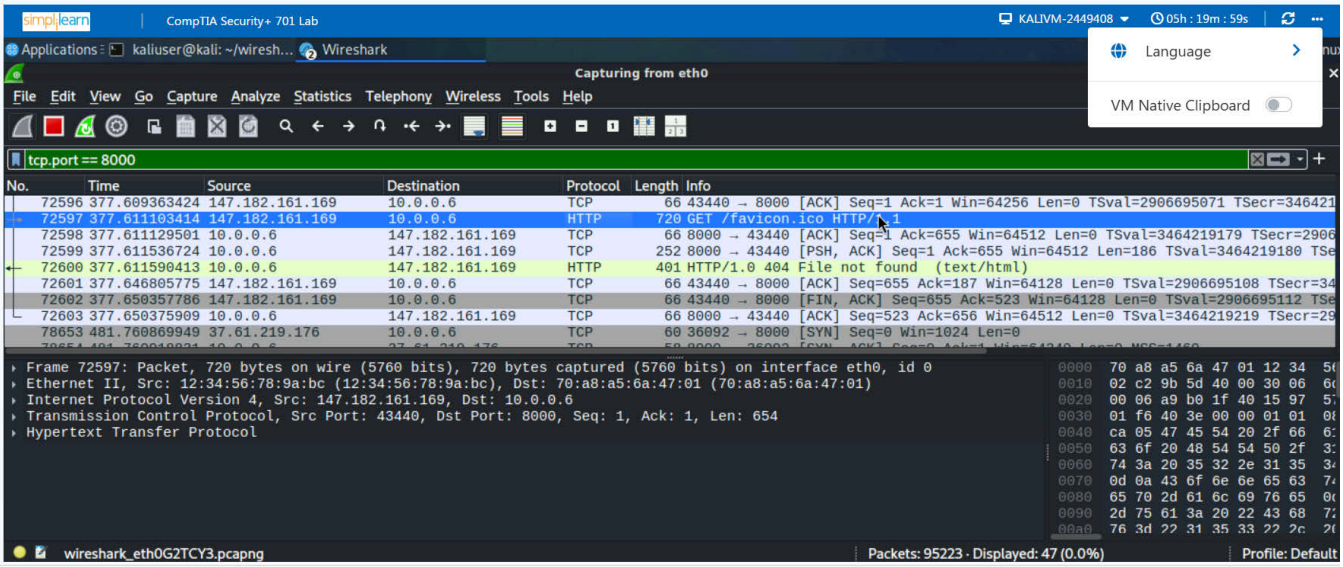
The PowerShell Test-NetConnection output records a failed TCP connection attempt to 10.0.0.6:8000 at that later point in the exercise. This is useful evidence that service reachability can change with service state, firewall rules, or lab timing; it does not contradict the earlier successful HTTP request.

Evidence 09: HTTP traffic and missing favicon



A later capture shows a browser request for /favicon.ico and an HTTP 404 File not found response. This is a missing optional website icon, not evidence that the main page itself failed.

Evidence 10: Final filtered packet capture



The final capture is filtered on TCP port 8000 and includes the HTTP request, server response, and TCP connection closure. The status bar shows 47 displayed packets in the selected capture at the time of the screenshot.